Zewail City of Science, Technology and Innovation

University of Science and Technology

School of Computational Sciences and Artificial Intelligence

CSAI 203 - Fall 2025

Introduction to Software Engineering

Software Design Document (SDD)

Project: Clinexa Medical System

Version 1.0

Team Number: 26

Email: s-ahmed.ragab@zewailcity.edu.eg

Team Members

Name	**ID**	**Email**
Ahmed Ragab	202400437	s-ahmed.ragab@zewailcity.edu.eg
Mosab Kamal	202402297	s-mosab.gabal@zewailcity.edu.eg
Mohamed Maher	202301798	s-mohammed.kafafy@zewailcity.edu.eg
Abdelrhman Khaled	202400161	s-abdelrhman.abdelhamid@zewailcity.edu.eg

Contents

1 Introduction **3**

1.1 Purpose of the Document . . . 3

1.2 Scope of the Design Phase . . . 3

1.3 Intended Audience . . . 4

1.4 Overview of the Contents . . . 5

2 System Overview **7**

2.1 Brief Description of the System . . . 7

2.2 Key Design Goals and Constraints . . . 7

2.2.1 Design Goals . . . 7

2.2.2 Constraints . . . 8

3 Architectural Design **10**

3.1 System Architecture Diagram . . . 10

3.1.1 Data Flow in the System . . . 12

3.2 Discussion of Architectural Style and Components . . . 13

3.2.1 Monolithic Architecture . . . 13

3.2.2 Component Descriptions . . . 15

3.3 Technology Stack and Tools . . . 16

3.3.1 Backend Technologies . . . 16

3.3.2 Frontend Technologies . . . 17

3.3.3 Data Storage . . . 17

3.3.4 Development Tools . . . 17

3.3.5 External Services . . . 18

3.3.6 Security Tools . . . 18

4 Detailed Design **20**

4.1 Model–View–Controller (MVC) Design Pattern . . . 20

4.1.1 Description of MVC Pattern . . . 20

4.1.2 Mapping of Project Components to MVC . . . 21

4.1.3 Responsibilities of Model, View, and Controller . . . 22

4.1.4 Interaction Between Components 24
4.2 UML Diagrams 26
4.2.1 Sequence Diagrams 26
4.3 UI/UX Design 31
4.3.1 Wireframes / Mockups 31
4.4 Data Design 36
4.4.1 Database Schema / ER Diagram (if applicable) 36
4.4.2 File Structure / Data Storage Model 37
4.4.3 File Schemas 38
4.4.4 Data Dictionary 40

5 Conclusion **47**
5.1 Summary of Design Decisions 47
5.2 Design Phase Outputs 49
5.3 Alignment with Requirements 50
5.4 Next Steps 51

1 Introduction

1.1 Purpose of the Document

This Software Design Document (SDD) defines the architectural and detailed design of the Clinexa Medical System based on the Software Requirements Specification (SRS). The document serves as a comprehensive blueprint that translates functional and non-functional requirements into a structured design that can be implemented by the development team.

The SDD provides:

- A clear explanation of the system's architectural structure and design patterns
- Detailed descriptions of system components and their interactions
- Visual representations through UML diagrams and UI mockups
- Data storage design and file structure specifications
- Technology stack and implementation guidelines

This document ensures that developers, testers, and stakeholders have a unified understanding of how the Clinexa system will be constructed, how its components will interact, and how it will meet the specified requirements.

1.2 Scope of the Design Phase

This design document covers the complete system design for the Clinexa Medical System, including:

- **Architectural Design:** High-level system architecture using a monolithic architectural style, component identification, and interaction patterns.
- **Design Patterns:** Implementation of the Model-View-Controller (MVC) design pattern to ensure separation of concerns and maintainability.
- **Module Design:** Detailed design of individual modules including User Management, Authentication, Patient Management, Doctor Management, Appointment Management, Medical Records, Notifications, Reports, and Search functionality.

- **Data Storage Design:** File-based data storage structure using text files with defined schemas, field separators, and parsing mechanisms.
- **User Interface Design:** Wireframes and mockups for key user interfaces including login pages, dashboards, appointment booking screens, and medical record views.
- **Component Interactions:** Sequence diagrams illustrating the flow of control and data between system components during key operations.

The design phase establishes a foundation for implementation that aligns with the project constraints: Flask backend, HTML/CSS frontend, file-based storage, and deployment on cloud servers accessible through modern web browsers.

1.3 Intended Audience

This Software Design Document is intended for multiple stakeholders involved in the Clinexa project:

- **Software Developers:** Primary audience who will implement the system according to the architectural and detailed design specifications provided in this document.
- **Quality Assurance Testers:** Will use this document to understand system structure, component interactions, and expected behaviors for creating comprehensive test plans.
- **Project Stakeholders:** Including project managers and healthcare administrators who need to review and approve the design approach before implementation begins.
- **Course Instructors and Evaluators:** Will assess the design quality, adherence to software engineering principles, and appropriate use of design patterns and UML diagrams.
- **Future Maintainers:** Developers who may need to maintain, extend, or modify the system after initial deployment.

1.4 Overview of the Contents

This Software Design Document is organized into the following sections to provide a comprehensive view of the Clinexa system design:

- **Section 1 – Introduction:** Defines the purpose of the design document, scope of the design phase, intended audience, and provides this structural overview.
- **Section 2 – System Overview:** Provides a high-level summary of the Clinexa Medical System, including its purpose, main features, design goals (usability, efficiency, reliability, maintainability, security), and constraints (technology stack, operating environment, external dependencies).
- **Section 3 – Architectural Design:** Presents the overall system architecture using a monolithic architectural style. Includes a system architecture diagram showing major components (Presentation Layer, Business Logic Layer, Data Access Layer), discusses the rationale for choosing this architecture, and details the technology stack (Flask, HTML/CSS, Python, file-based storage).
- **Section 4 – Detailed Design:** The most comprehensive section containing:
  - **4.1 Model-View-Controller (MVC) Design Pattern:** Explains the MVC pattern, maps Clinexa components to MVC layers, defines responsibilities, and illustrates component interactions.
  - **4.2 UML Diagrams:** Provides sequence diagrams for key operations (login, appointment booking, medical record management).
  - **4.3 UI/UX Design:** Presents wireframes and mockups for major user interfaces including login screens, patient dashboards, doctor dashboards, appointment booking interfaces, and medical record views.
  - **4.4 Data Design:** Details the file-based storage structure with complete schemas for all nine data files (users, patients, doctors, appointments, medical records, notifications, authentication logs, feedback, reports), including field definitions, data types, separators, and example data.

- **Section 5 – Conclusion:** Summarizes the key design decisions, outputs from the design phase, and confirms that the design meets all requirements specified in the SRS.

This structured approach ensures comprehensive coverage of all design aspects necessary for successful implementation of the Clinexa Medical System.

2 System Overview

2.1 Brief Description of the System

Clinexa is a web-based healthcare management system designed to modernize and streamline the interaction between healthcare providers and patients. The system replaces traditional paper-based appointment scheduling and medical record management with a comprehensive digital platform accessible through standard web browsers.

The platform serves two primary user groups: patients and doctors. Patients can create accounts, search for available doctors by specialization, book appointments based on doctor availability, view their complete medical history including past diagnoses and prescriptions, receive automated reminders for upcoming appointments via email and SMS, cancel or reschedule appointments when necessary, and submit feedback and ratings after consultations. Doctors can manage their availability schedules, view daily appointment lists with patient information, access patient medical histories before consultations, create and update medical records with diagnoses and prescriptions after appointments, generate various reports including workload analysis and patient visit summaries, and respond to patient feedback.

The system aims to improve healthcare service delivery by reducing administrative overhead, minimizing scheduling conflicts through automated availability checking, ensuring secure and organized medical record storage, facilitating timely communication through automated notifications, and providing analytics through report generation capabilities. By digitizing these processes, Clinexa enhances efficiency for healthcare providers while improving accessibility and convenience for patients seeking medical care.

2.2 Key Design Goals and Constraints

2.2.1 Design Goals

The Clinexa system design is guided by the following key goals that align with both functional requirements and quality attributes:

- **Usability:** The system must provide an intuitive, user-friendly interface that requires minimal training for both patients and doctors. Navigation should be

straightforward, with clear labels and logical workflow progression. The interface must be responsive and accessible through modern web browsers (Chrome, Firefox) on various devices including desktops, laptops, and tablets.

- **Efficiency:** Core processes such as appointment scheduling, medical record retrieval, and notification delivery must be streamlined to minimize the time required to complete tasks. The system should reduce the administrative burden on healthcare staff by automating routine tasks like appointment reminders and availability checking.

- **Reliability:** The system must maintain 99.9% uptime to ensure continuous availability for scheduling appointments and accessing medical records. In the event of system failures, recovery time must not exceed 5 minutes to minimize disruption to healthcare services. Data integrity must be maintained through proper validation and error handling.

- **Maintainability:** The system architecture must support easy updates and modifications with a maintenance success rate of 90% and maintenance speed of 95%. Clear separation of concerns through the MVC pattern, well-documented code, and modular design facilitate ongoing maintenance and feature additions.

- **Security:** The system must implement robust authentication mechanisms including password protection, login attempt tracking, and automatic account locking after three failed attempts. Role-based access control ensures users can only access features and data appropriate to their role (patient, doctor, admin). Medical records and personal information must be protected according to healthcare data privacy standards.

- **Scalability:** While initially designed as a monolithic application, the architecture should allow for future growth in terms of user base, data volume, and feature additions without requiring complete system redesign.

2.2.2 Constraints

The system design must operate within several technical and operational constraints defined in the Software Requirements Specification:

- **Technology Stack Constraints:**
  - Backend framework: Flask (Python web framework)
  - Frontend technologies: HTML and CSS (no JavaScript frameworks)
  - Data storage: File-based system using text files with structured formats
  - Diagram notation: UML 2.5 standard
  - Diagramming tools: draw.io or Lucidchart
- **Operating Environment Constraints:**
  - Client-side: Must run in Chrome and Firefox web browsers
  - Server-side: Hosted on cloud server infrastructure running Flask application
  - No special hardware requirements beyond standard computing devices
  - Requires stable internet connection for client-server communication
- **External Dependencies:**
  - Email/SMS gateway API for sending notifications and reminders
  - HTTP/HTTPS protocols for secure client-server communication
  - SMTP protocol for email delivery
  - Users must possess basic computer literacy and web browser navigation skills
- **Development Constraints:**
  - Academic project timeline and milestones
  - Team collaboration requirements
  - Limited to technologies covered in the course curriculum
  - Budget constraints typical of student projects

These constraints shape architectural decisions and implementation approaches throughout the design process, ensuring the final system is feasible within project limitations while meeting all functional requirements.

3 Architectural Design

3.1 System Architecture Diagram

The Clinexa Medical System follows a three-tier monolithic architecture, organizing the system into three distinct layers that work together to provide complete healthcare management functionality. Figure 1 illustrates the overall system architecture.

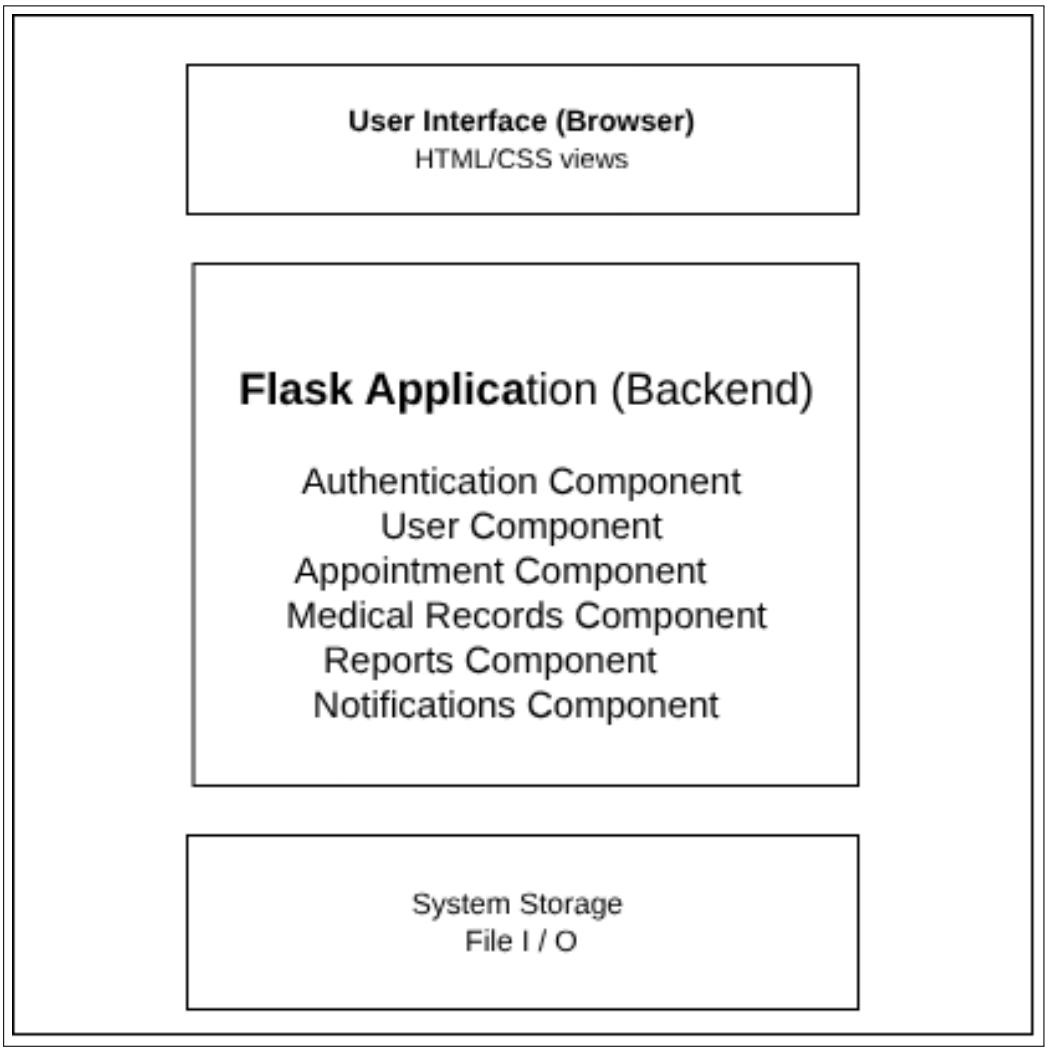


Figure 1: Clinexa System Architecture - Three-Tier Monolithic Design showing Presentation Layer (HTML/CSS), Business Logic Layer (Flask), and Data Access Layer (File System)

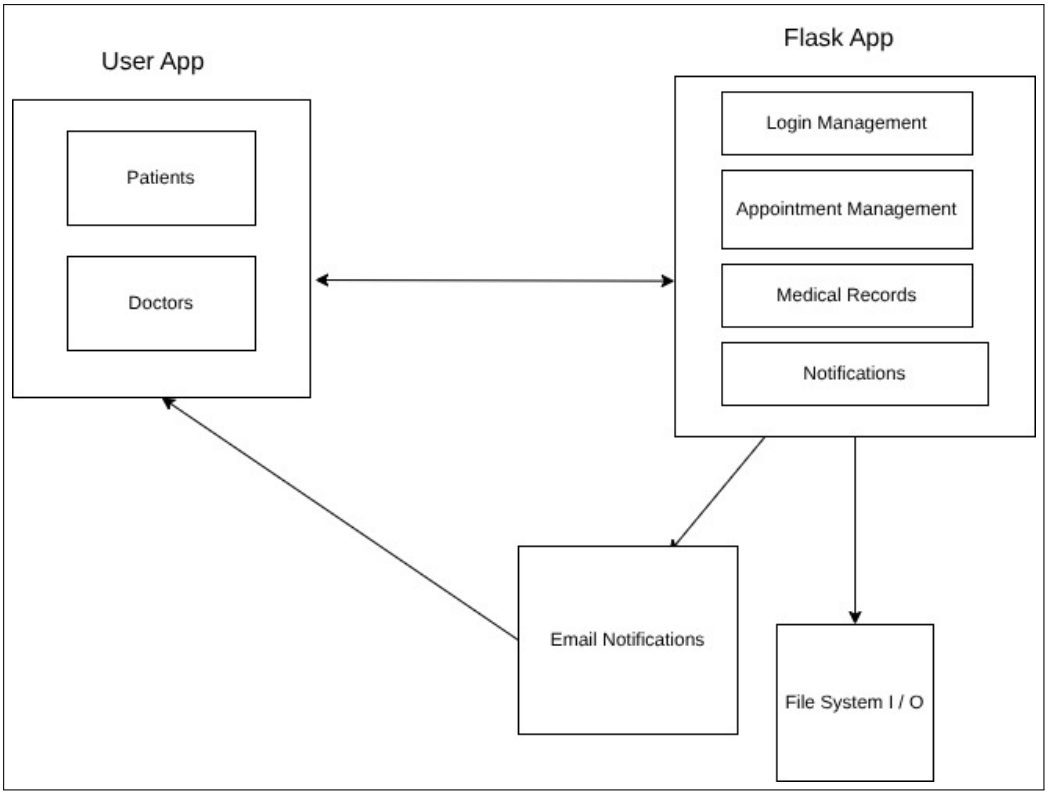


Figure 2: Detailed Architecture View - Component breakdown of the Clinexa monolithic architecture with module interactions

The architecture consists of the following layers:

1. **Presentation Layer (Frontend):**
   - Web Browser Interface (HTML/CSS)
   - User interaction components (forms, buttons, navigation)
   - Login pages, dashboards, appointment booking screens
   - Medical record displays and report views
   - Responsive design for various screen sizes
2. **Business Logic Layer (Backend):**
   - Flask Web Application Server
   - Controllers for request handling and routing
   - Business logic for appointments, medical records, notifications
   - Authentication and authorization services
   - Validation and error handling

- Report generation and search functionality

3. **Data Access Layer:**
   - File system management for text-based storage
   - Data reading/writing utilities
   - Data parsing and formatting functions
   - File locking mechanisms for data integrity
   - Backup and recovery utilities

3.1.1 Data Flow in the System

Understanding how data flows through the Clinexa system is essential for comprehending the interaction between architectural layers. The following describes the typical data flow for user operations:

1. **User Interaction → Browser Request:**
   - User interacts with the web interface (clicks button, submits form)
   - Browser collects form data and creates HTTP request
   - Request is sent to Flask server with appropriate route and parameters
2. **Flask Processing → File Operations:**
   - Flask receives the request at a specific route/endpoint
   - Controller validates input data and checks user permissions
   - If data retrieval is needed, Controller calls Model methods
   - Model interacts with Data Access Layer to read from text files
   - Data is parsed from pipe-separated format into Python objects
   - If data modification is needed, Model validates and writes to files
   - Business logic is applied (e.g., checking appointment availability)
3. **Flask Response → Webpage Delivery:**
   - Controller prepares response data based on operation results

- Flask renders appropriate HTML template with data
- Response is sent back to browser as complete webpage
- Browser displays the updated page to the user

4. **External Notifications (When Needed):**
   - If notification is required (appointment confirmation, reminder)
   - Notification Controller triggers email/SMS gateway
   - SMTP protocol sends emails to user's registered address
   - SMS API sends text messages to user's phone number
   - Notification status is logged in notifications.txt file

Example Data Flow - Booking an Appointment:

1. Patient selects doctor and time slot → Browser sends POST request
2. Flask AppointmentController receives booking request
3. Controller checks doctor availability by reading appointments.txt
4. If available, new appointment record is written to appointments.txt
5. Notification is sent via email/SMS to both patient and doctor
6. Success page is rendered and sent back to browser
7. Patient sees confirmation message with appointment details

Communication between layers follows a unidirectional flow pattern, ensuring clear separation of responsibilities and making the system easier to understand, test, and maintain.

3.2 Discussion of Architectural Style and Components

3.2.1 Monolithic Architecture

Clinexa adopts a monolithic architectural style where all system components are integrated into a single, unified application. This architecture was selected based on several project-specific factors:

Rationale for Monolithic Architecture:

- **Project Scale:** The system scope is well-defined and manageable as a single application without requiring the complexity of distributed systems or microservices.
- **Team Structure:** A four-member student team benefits from working on a cohesive codebase rather than managing multiple independent services with complex inter-service communication.
- **Development Simplicity:** Monolithic architecture simplifies development, testing, and deployment processes, which is essential for an academic project with time constraints.
- **Technology Constraints:** Flask is optimized for monolithic web applications, and the file-based storage system works efficiently within a single application context.
- **Deployment Ease:** A monolithic application can be deployed as a single unit on a cloud server without requiring orchestration tools or complex infrastructure.
- **Performance:** For the expected user load and data volume, a monolithic architecture provides excellent performance without the network overhead of distributed systems.

Advantages:

- Simplified development and debugging
- Easier testing with all components in one codebase
- Straightforward deployment process
- Better performance for moderate scale
- Lower operational complexity

Potential Limitations and Mitigation:

- *Scalability:* While monolithic applications can face scaling challenges, the MVC pattern and modular design within the monolith allow for future refactoring if needed.
- *Technology Lock-in:* The entire application uses Python/Flask, but clear separation of concerns through MVC makes it possible to migrate specific components if necessary.

3.2.2 Component Descriptions

1. Presentation Layer Components:

- **User Interface Views:** HTML templates rendered by Flask that present data to users and collect user input through forms. Views are role-specific (patient views, doctor views, admin views).
- **Styling and Layout:** CSS files that define the visual presentation, ensuring consistent branding and user experience across all pages.
- **Client-Side Validation:** Basic HTML5 form validation to improve user experience by catching input errors before submission to the server.

2. Business Logic Layer Components:

- **Authentication Module:** Handles user login, logout, password validation, failed login attempt tracking, and account blocking. Manages user sessions and role-based access control.
- **User Management Module:** Manages user profiles, registration, and profile updates for both patients and doctors.
- **Appointment Management Module:** Implements appointment scheduling logic, availability checking, double-booking prevention, cancellation handling, and rescheduling functionality.
- **Medical Records Module:** Manages creation, retrieval, and updates of patient medical records including diagnoses, prescriptions, and vital signs.
- **Notification Module:** Generates and sends email and SMS notifications for appointment reminders, confirmations, and cancellations.
- **Report Generation Module:** Creates various reports including daily appointments, patient history, and doctor workload analysis.
- **Search Module:** Implements search functionality for patients, appointments, and medical records with partial matching capabilities.

3. Data Access Layer Components:

- **File Management Utilities:** Functions for reading from and writing to text files, handling file locking, and ensuring data consistency.
- **Data Parsing Module:** Converts structured text data (pipe-separated values) into Python objects and vice versa.
- **Data Validation:** Ensures data integrity before writing to files, including type checking, format validation, and constraint verification.
- **Backup Module:** Creates periodic backups of data files to prevent data loss.

3.3 Technology Stack and Tools

3.3.1 Backend Technologies

- **Programming Language:** Python 3.x
  - Chosen for its readability, extensive libraries, and suitability for web development
  - Strong support for file operations and text processing
  - Wide adoption in the development community with abundant resources
- **Web Framework:** Flask 2.x
  - Lightweight and flexible micro-framework ideal for learning and rapid development
  - Supports MVC pattern through blueprints and templates
  - Built-in development server for testing
  - Excellent documentation and community support
  - Easy deployment to cloud platforms
- **Template Engine:** Jinja2 (integrated with Flask)
  - Powerful template language for dynamic HTML generation
  - Supports template inheritance and reusable components
  - Security features like automatic escaping to prevent XSS attacks

3.3.2 Frontend Technologies

- **Markup Language:** HTML5
  - Modern semantic elements for better structure
  - Built-in form validation attributes
  - Accessibility features for inclusive design
- **Styling:** CSS3
  - Flexbox and Grid layouts for responsive design
  - Custom styling for professional appearance
  - Media queries for different screen sizes

3.3.3 Data Storage

- **Storage Mechanism:** File-based text storage
  - Plain text files (.txt) with structured data format
  - Pipe-separated values (PSV) for data fields
  - UTF-8 encoding for international character support
  - Simple backup and version control integration

3.3.4 Development Tools

- **Integrated Development Environment (IDE):**
  - PyCharm, Visual Studio Code, or similar Python IDEs
  - Code completion and debugging support
  - Integrated terminal for running Flask applications
- **Version Control:** Git and GitHub
  - Source code versioning and collaboration
  - Branch management for parallel development
  - Issue tracking and project management

- **Diagramming Tools:**
  - draw.io or Lucidchart for UML diagrams
  - UML 2.5 notation standards
  - Export to PNG/PDF for documentation
- **Testing Tools:**
  - pytest for unit and integration testing
  - Flask testing utilities for endpoint testing
  - Manual testing through web browsers

3.3.5 External Services

- **Email Service:** SMTP-compatible email service
  - For sending appointment reminders and notifications
  - Support for HTML-formatted emails
- **SMS Gateway:** Third-party SMS API
  - For sending text message notifications
  - Integration through REST API
- **Hosting:** Cloud hosting platform
  - Heroku, PythonAnywhere, or similar Flask-compatible platforms
  - SSL/TLS support for HTTPS
  - Automatic backups and monitoring

3.3.6 Security Tools

- **Password Hashing:** Werkzeug security utilities
  - Secure password hashing with salt
  - Protection against rainbow table attacks
- **Session Management:** Flask-Session

- Secure session handling
- Session timeout management

This technology stack provides a solid foundation for developing a secure, maintainable, and scalable healthcare management system while adhering to project constraints and leveraging technologies appropriate for an academic software engineering project.

4 Detailed Design

4.1 Model–View–Controller (MVC) Design Pattern

4.1.1 Description of MVC Pattern

The Model-View-Controller (MVC) is a widely adopted architectural design pattern that separates an application into three interconnected components, each with distinct responsibilities. This separation promotes organized code structure, maintainability, and scalability—critical factors for healthcare systems that must be reliable and easy to maintain.

Components of MVC:

- **Model:** Represents the application's data layer and business logic. It manages the system's data, rules, and operations independent of the user interface. The Model is responsible for data validation, business rule enforcement, and interaction with the data storage layer.
- **View:** Responsible for presenting data to users and rendering the user interface. It displays information from the Model in a format suitable for interaction, without containing business logic or directly manipulating data.
- **Controller:** Acts as an intermediary between the Model and View. It processes user input from the View, invokes appropriate Model operations, and determines which View to display based on the results.

Why MVC Was Chosen for Clinexa:

1. **Separation of Concerns:** MVC clearly separates data management (Model), presentation logic (View), and application flow control (Controller). This makes the codebase more organized and easier to understand—essential for a medical system handling sensitive patient data and complex workflows.
2. **Maintainability:** With distinct boundaries between components, updates to the user interface do not affect business logic, and changes to data structures do not require modifications to the presentation layer. This is critical for Clinexa's long-term maintenance and regulatory compliance requirements.

3. **Parallel Development:** Different team members can work simultaneously on Models, Views, and Controllers without significant conflicts, improving development efficiency for Team 26's collaborative workflow.

4. **Testability:** Each component can be tested independently. Business logic in Models can be unit tested without rendering Views, and Controllers can be tested with mock Models and Views, ensuring reliability for the 99.9% uptime requirement specified in the SRS.

5. **Framework Compatibility:** Flask naturally supports the MVC pattern through its routing mechanisms (Controller), template rendering (View), and Python classes/modules (Model), making implementation straightforward and aligned with industry best practices.

6. **Scalability:** As Clinexa grows to include additional features such as telemedicine, prescription management, or integration with external medical systems, the MVC structure allows new functionality to be added with minimal disruption to existing code.

4.1.2 Mapping of Project Components to MVC

The Clinexa system maps its nine major functional components to the MVC pattern. Each component has its own Model, View, and Controller, creating a modular structure. Table 1 provides a comprehensive mapping.

Table 1: Mapping of Clinexa Components to MVC Pattern

Component	**Model**	**View**	**Controller**
User Management	User class with attributes (userID, username, password, name, email, phone, role) and methods (login(), logout(), updateProfile())	UserLoginView, UserProfileView with forms and displays	UserController handles login, logout, profile updates, session management
Authentication	Authentication class tracking login attempts, blocking status, tokens	AuthenticationView with login page, error messages, password reset forms	AuthController processes login, validates credentials, manages blocking, handles password reset
Patient Management	Patient class with medical history, demographics, emergency contacts	PatientDashboardView, PatientProfileView showing appointments and records	PatientController manages registration, appointments, medical records access
Doctor Management	Doctor class with specialization, availability schedule, consultation fees	DoctorDashboardView, DoctorScheduleView showing appointments and patient lists	DoctorController manages schedules, patient records access, appointment confirmations
Appointment Management	Appointment class with date, time, status, patient/doctor IDs, meeting links	AppointmentBookingView, AppointmentListView with booking forms and appointment lists	AppointmentController processes bookings, cancellations, rescheduling, availability checking
Medical Records	AppointmentRecord class with diagnoses, prescriptions, vital signs, notes	MedicalRecordView with record forms and history displays	RecordController creates/updates records, retrieves patient history
Notification System	Notification class with message, recipient, type, status, channel	NotificationView with notification lists and alerts	NotificationController sends reminders, confirmations, manages notification queue
Report Generation	Report class with report type, parameters, generated data	ReportView with filters, charts, statistics, PDF previews	ReportController processes requests, collects data, formats reports, handles exports
Search Functionality	Search class with query, filters, results, search type	SearchView with search bar, filters, result displays	SearchController processes queries, applies filters, sorts and paginates results

4.1.3 Responsibilities of Model, View, and Controller

Each component of the MVC pattern has specific responsibilities that ensure proper separation of concerns and maintainable code structure.

Model Responsibilities:

- **Data Representation:** Define the structure of domain objects (User, Patient, Doctor, Appointment, etc.) with appropriate attributes and data types.
- **Business Logic:** Implement core business rules such as appointment availability checking, double-booking prevention, and medical record validation.
- **Data Persistence:** Interact with the Data Access Layer to read from and write to text files, ensuring data is properly formatted and stored.
- **Data Validation:** Validate input data before processing, including format checking, constraint verification, and business rule compliance.
- **State Management:** Maintain the current state of domain objects and handle state transitions (e.g., appointment status changes from scheduled to completed).
- **Calculations and Processing:** Perform computations such as report generation, search result filtering, and notification scheduling.

View Responsibilities:

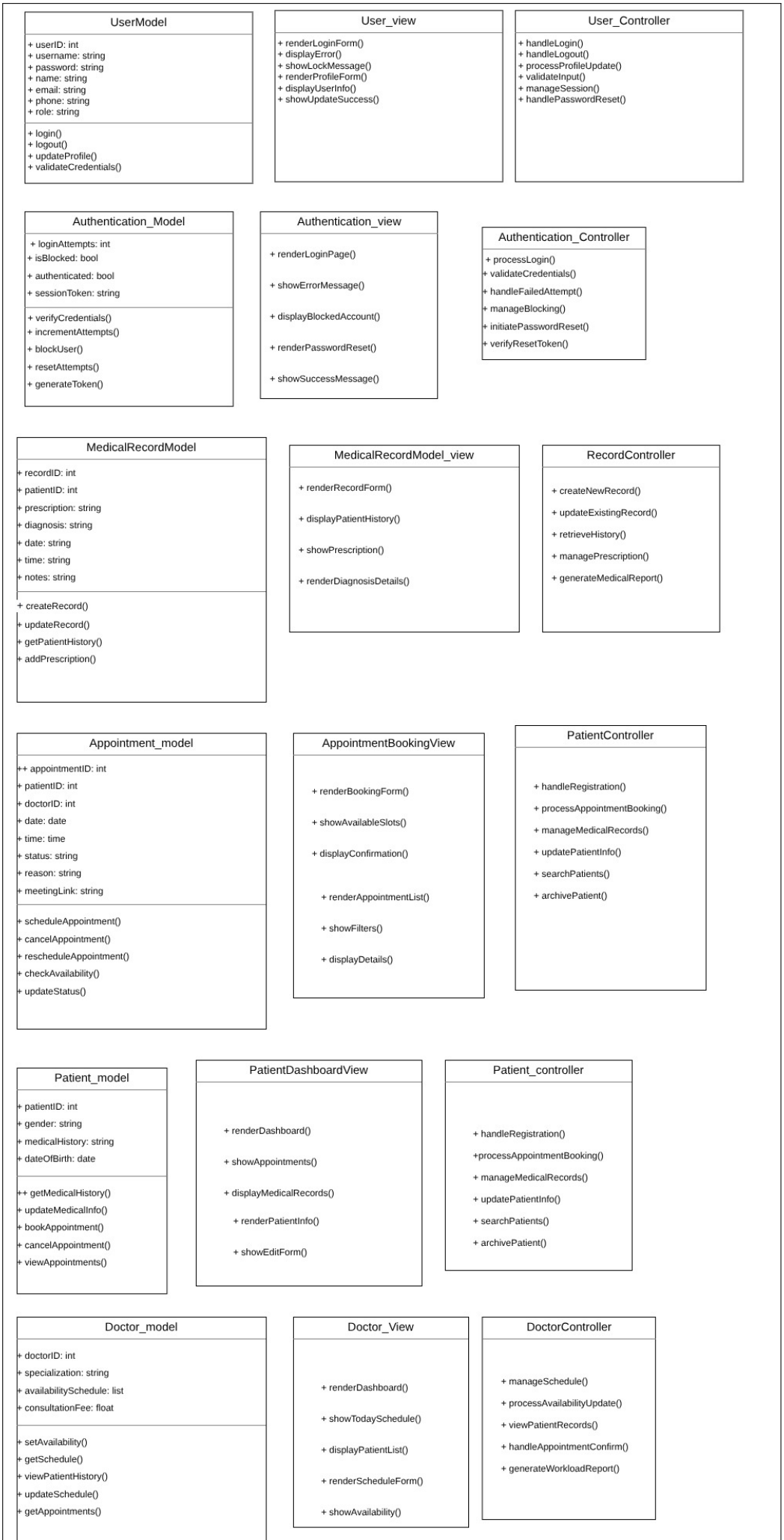
- **Data Presentation:** Display information from Models in a user-friendly format using HTML templates with proper formatting and styling.
- **User Input Collection:** Provide forms and interactive elements for users to input data and trigger actions.
- **Visual Feedback:** Show success messages, error notifications, and loading indicators to keep users informed of system status.
- **Template Rendering:** Use Jinja2 templates to dynamically generate HTML based on data passed from Controllers.
- **Responsive Design:** Ensure interfaces work properly across different screen sizes and devices.
- **Accessibility:** Implement proper semantic HTML and ARIA attributes for users with disabilities.

Controller Responsibilities:

- **Request Handling:** Receive and parse HTTP requests from the browser, extracting parameters and form data.
- **Input Validation:** Perform initial validation of user input before passing to Models.
- **Business Flow Control:** Orchestrate the sequence of operations needed to fulfill a request, calling multiple Models if necessary.
- **Authentication and Authorization:** Verify user identity and check permissions before allowing access to protected resources.
- **Model Invocation:** Call appropriate Model methods to perform business logic and data operations.
- **View Selection:** Determine which View template to render based on the operation results and user role.
- **Response Generation:** Prepare data for the View and generate HTTP responses to send back to the browser.
- **Error Handling:** Catch exceptions from Models and present appropriate error messages to users.

4.1.4 Interaction Between Components

Figure 3 illustrates how the Model, View, and Controller components interact during a typical user operation in the Clinexa system.


UserModel
+ userID: int
+ username: string
+ password: string
+ name: string
+ email: string
+ phone: string
+ role: string
+ login()
+ logout()
+ updateProfile()
+ validateCredentials()
User_view
+ renderLoginForm()
+ displayError()
+ showLockMessage()
+ renderProfileForm()
+ displayUserInfo()
+ showUpdateSuccess()
User_Controller
+ handleLogin()
+ handleLogout()
+ processProfileUpdate()
+ validateInput()
+ manageSession()
+ handlePasswordReset()
Authentication_Model
+ loginAttempts: int
+ isBlocked: bool
+ authenticated: bool
+ sessionToken: string
+ verifyCredentials()
+ incrementAttempts()
+ blockUser()
+ resetAttempts()
+ generateToken()
Authentication_view
+ renderLoginPage()
+ showErrorMessage()
+ displayBlockedAccount()
+ renderPasswordReset()
+ showSuccessMessage()
Authentication_Controller
+ processLogin()
+ validateCredentials()
+ handleFailedAttempt()
+ manageBlocking()
+ initiatePasswordReset()
+ verifyResetToken()
MedicalRecordModel
+ recordID: int
+ patientID: int
+ prescription: string
+ diagnosis: string
+ date: string
+ time: string
+ notes: string
+ createRecord()
+ updateRecord()
+ getPatientHistory()
+ addPrescription()
MedicalRecordModel_view
+ renderRecordForm()
+ displayPatientHistory()
+ showPrescription()
+ renderDiagnosisDetails()
RecordController
+ createNewRecord()
+ updateExistingRecord()
+ retrieveHistory()
+ managePrescription()
+ generateMedicalReport()
Appointment_model
++ appointmentID: int
+ patientID: int
+ doctorID: int
+ date: date
+ time: time
+ status: string
+ reason: string
+ meetingLink: string
+ scheduleAppointment()
+ cancelAppointment()
+ rescheduleAppointment()
+ checkAvailability()
+ updateStatus()
AppointmentBookingView
+ renderBookingForm()
+ showAvailableSlots()
+ displayConfirmation()
+ renderAppointmentList()
+ showFilters()
+ displayDetails()
PatientController
+ handleRegistration()
+ processAppointmentBooking()
+ manageMedicalRecords()
+ updatePatientInfo()
+ searchPatients()
+ archivePatient()
Patient_model
+ patientID: int
+ gender: string
+ medicalHistory: string
+ dateOfBirth: date
++ getMedicalHistory()
+ updateMedicalInfo()
+ bookAppointment()
+ cancelAppointment()
+ viewAppointments()
PatientDashboardView
+ renderDashboard()
+ showAppointments()
+ displayMedicalRecords()
+ renderPatientInfo()
+ showEditForm()
Patient_controller
+ handleRegistration()
+processAppointmentBooking()
+ manageMedicalRecords()
+ updatePatientInfo()
+ searchPatients()
+ archivePatient()
Doctor_model
+ doctorID: int
+ specialization: string
+ availabilitySchedule: list
+ consultationFee: float
+ setAvailability()
+ getSchedule()
+ viewPatientHistory()
+ updateSchedule()
+ getAppointments()
Doctor_View
+ renderDashboard()
+ showTodaySchedule()
+ displayPatientList()
+ renderScheduleForm()
+ showAvailability()
DoctorController
+ manageSchedule()
+ processAvailabilityUpdate()
+ viewPatientRecords()
+ handleAppointmentConfirm()
+ generateWorkloadReport()

Typical Interaction Flow:

1. **User Action:** User interacts with the View (e.g., submits appointment booking form)
2. **View to Controller:** View sends user input to appropriate Controller via HTTP request
3. **Controller Processing:** Controller validates input and determines which Model methods to call
4. **Controller to Model:** Controller invokes Model methods with validated data
5. **Model Operations:** Model performs business logic and interacts with data files
6. **Model to Controller:** Model returns results (success/failure, data objects, error messages)
7. **Controller to View:** Controller selects appropriate View and passes data for rendering
8. **View Rendering:** View generates HTML with the data and sends response to browser
9. **User Feedback:** User sees the updated interface with results of their action

4.2 UML Diagrams

4.2.1 Sequence Diagrams

Sequence diagrams illustrate the interaction flow over time for key system operations. These diagrams show how objects collaborate to achieve specific functionality.

1. User Login Sequence

Figure 4 shows the sequence of interactions when a user attempts to log into the system, including authentication validation and session creation.

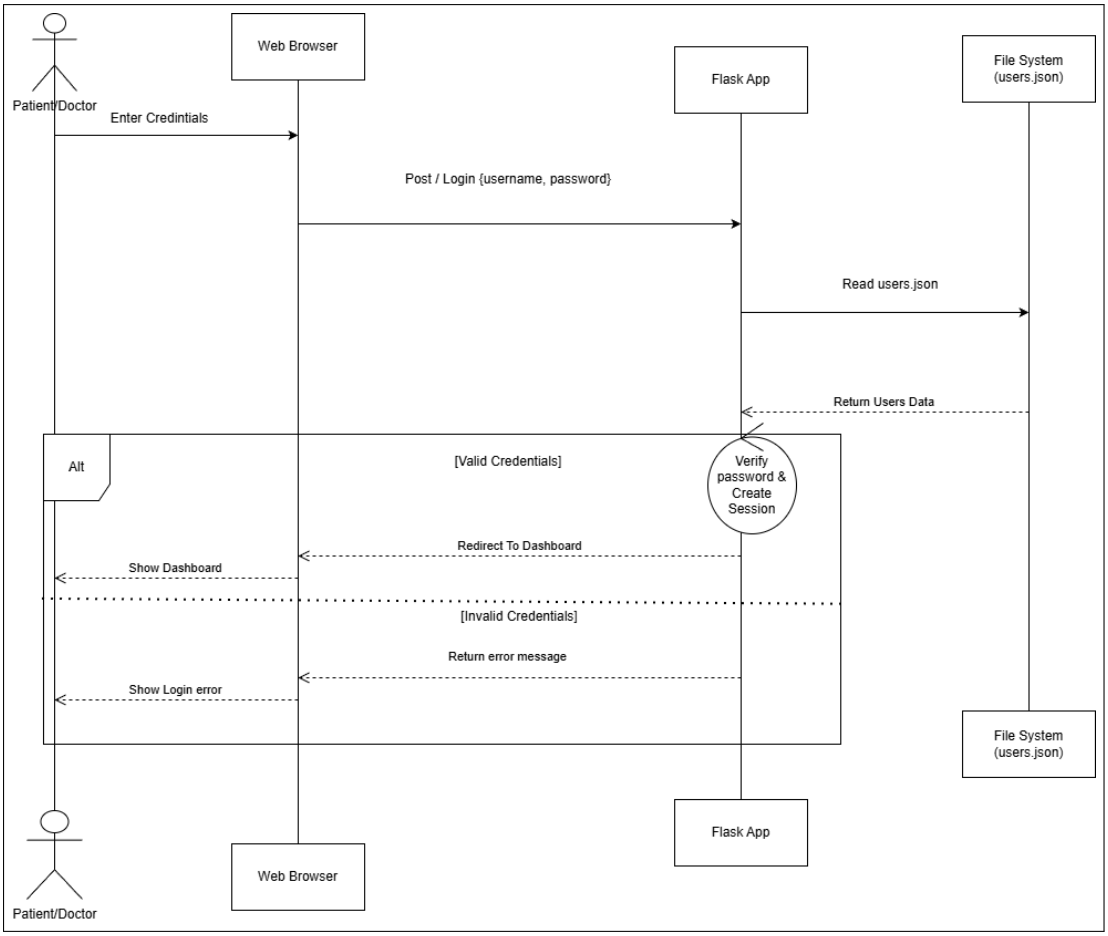


Figure 4: Login Sequence Diagram - Shows the flow from user entering credentials through authentication validation, failed attempt tracking, account blocking logic, and session creation upon successful login

2. Appointment Booking Sequence

Figure 5 illustrates the complete process of booking an appointment, from checking doctor availability to sending confirmation notifications.

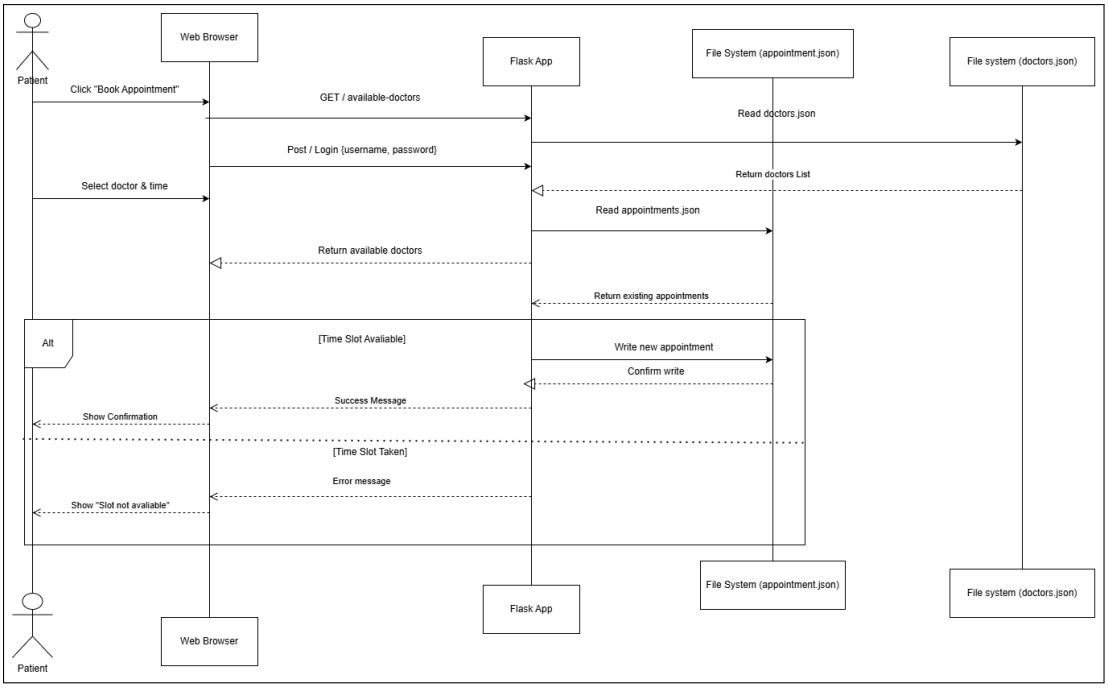


Figure 5: Appointment Booking Sequence Diagram - Shows patient selecting doctor and time slot, availability checking, double-booking prevention, appointment creation, and notification delivery to both patient and doctor

3. Medical Record Management Sequence

Figure 6 depicts how doctors create and update medical records after patient consultations, including prescription management.

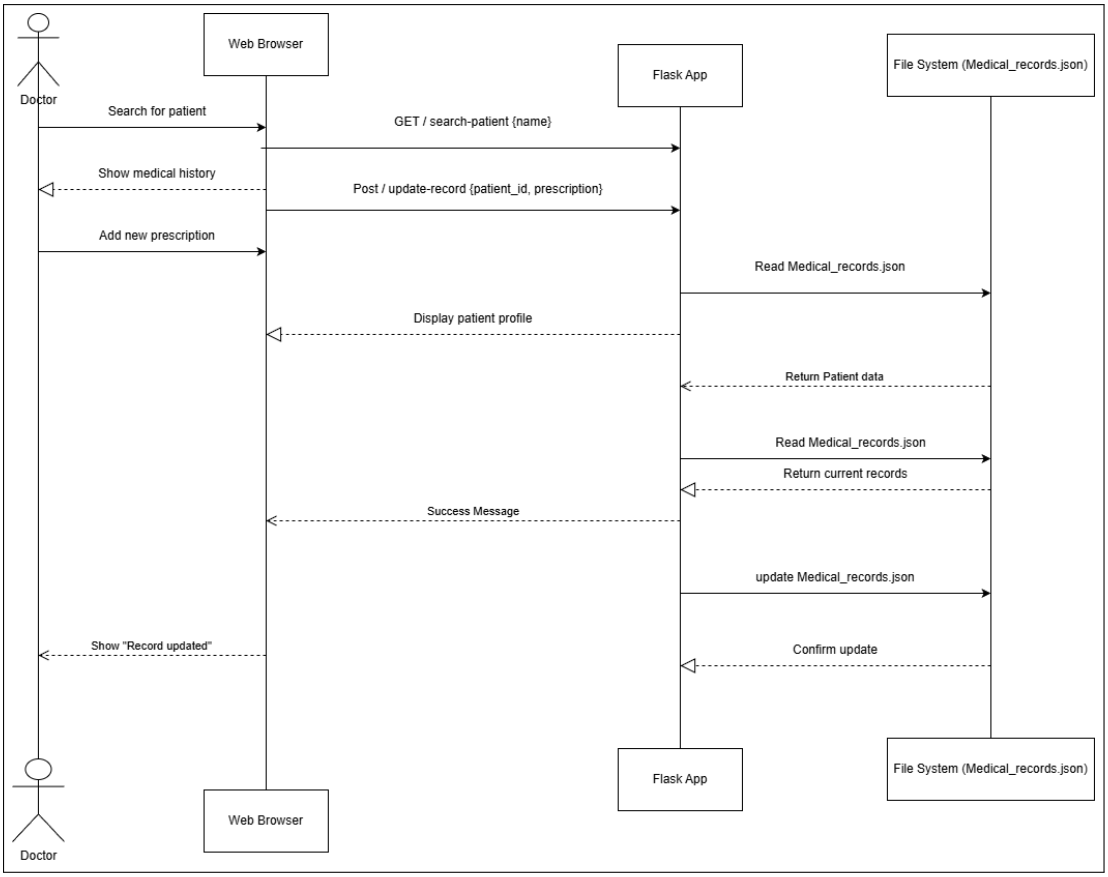


Figure 6: Medical Record Management Sequence Diagram - Shows doctor accessing patient history, conducting consultation, entering diagnosis and prescriptions, updating vital signs, and saving the complete medical record

4. Report Generation Sequence

Figure 7 shows the process of generating various reports including data collection, aggregation, and export functionality.

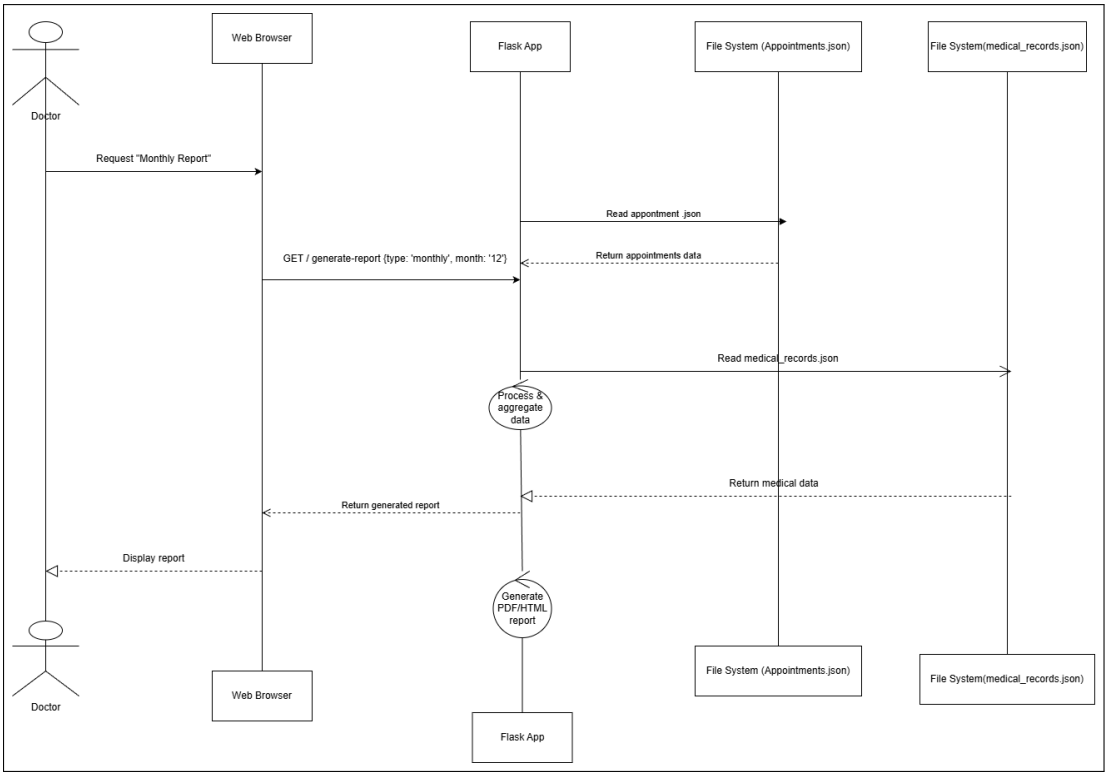


Figure 7: Report Generation Sequence Diagram - Shows user selecting report type and parameters, data collection from multiple files, report formatting, and export to PDF format

5. Search and Filter Sequence

Figure 8 demonstrates the search functionality for finding doctors, patients, or appointments with filtering capabilities.

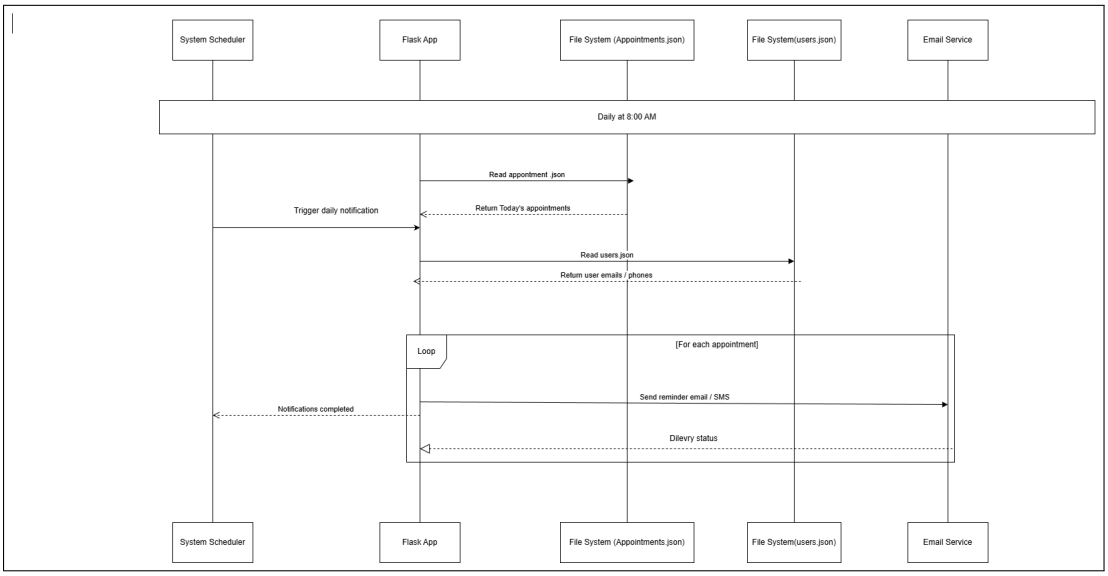


Figure 8: Search and Filter Sequence Diagram - Shows user entering search query, applying filters, system processing search criteria, and returning filtered results with pagination

4.3 UI/UX Design

4.3.1 Wireframes / Mockups

This section presents wireframes and mockups for the major user interfaces in the Clinexa system. These designs illustrate the layout, navigation, and key features of each screen.

1. Login Page

Figure 9 shows the login page design with username/password fields, error message display, and password reset option.

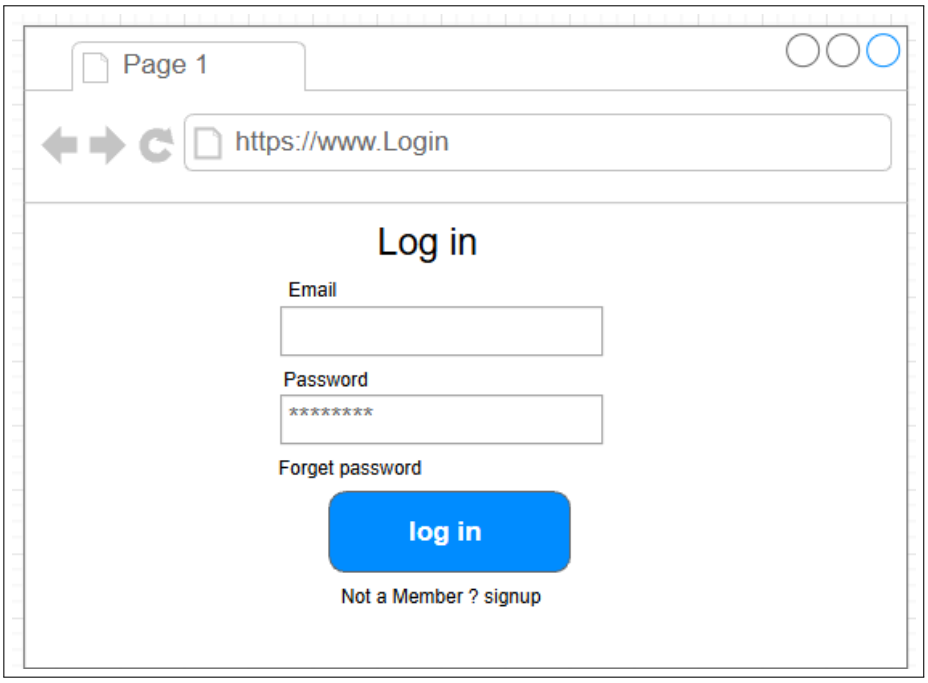


Figure 9: Login Page Wireframe - Shows login form with username and password fields, remember me option, forgot password link, and error message display area for invalid credentials or locked accounts

2. Patient Dashboard

Figure 10 presents the patient dashboard showing upcoming appointments, quick actions, and recent medical records.

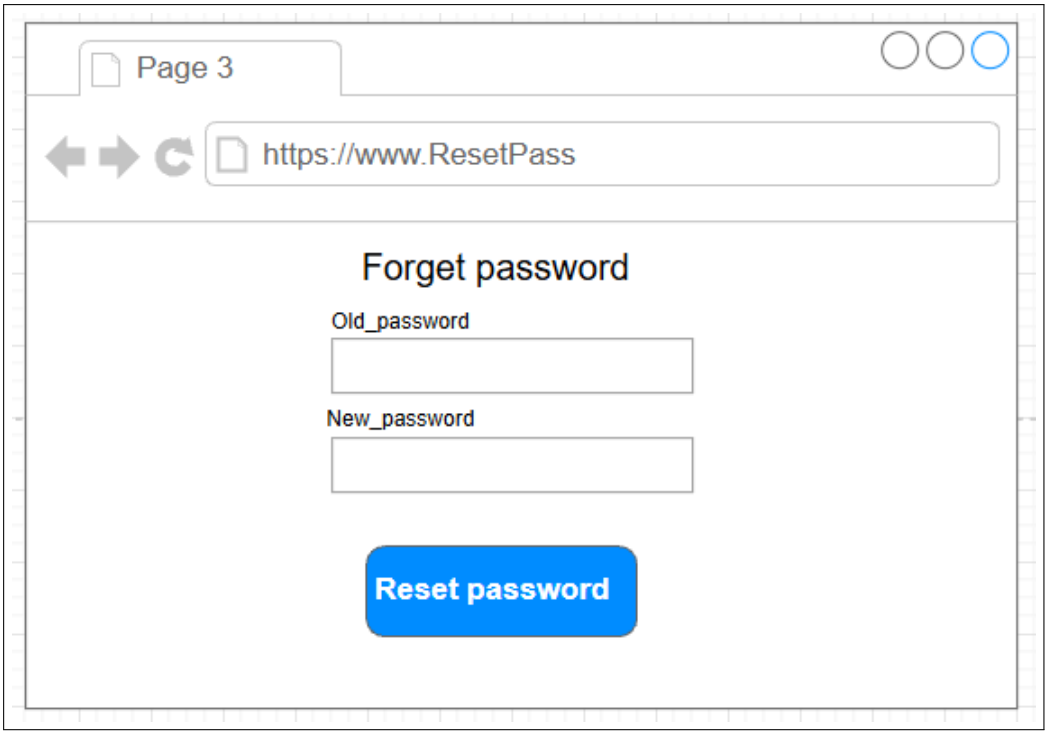


Figure 10: Patient Dashboard Wireframe - Displays upcoming appointments list, quick action buttons for booking new appointments, viewing medical history, and accessing profile settings

3. Doctor Dashboard

Figure 11 illustrates the doctor dashboard with today's appointment schedule, patient list, and quick access to medical records.

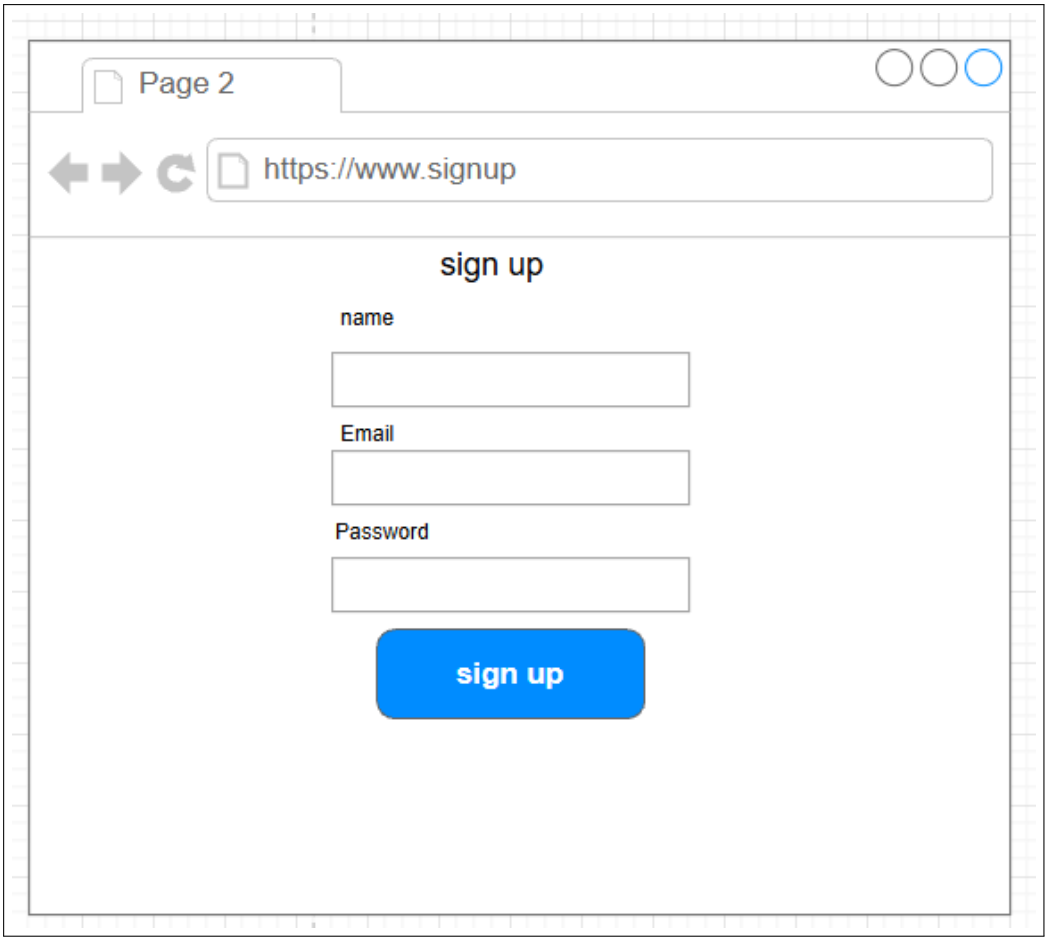


Figure 11: Doctor Dashboard Wireframe - Shows today's appointment schedule with patient names and times, patient list with quick access to medical histories, and options to manage availability and generate reports

4. Appointment Booking Page

Figure 12 shows the appointment booking interface with doctor selection, date/time picker, and availability display.

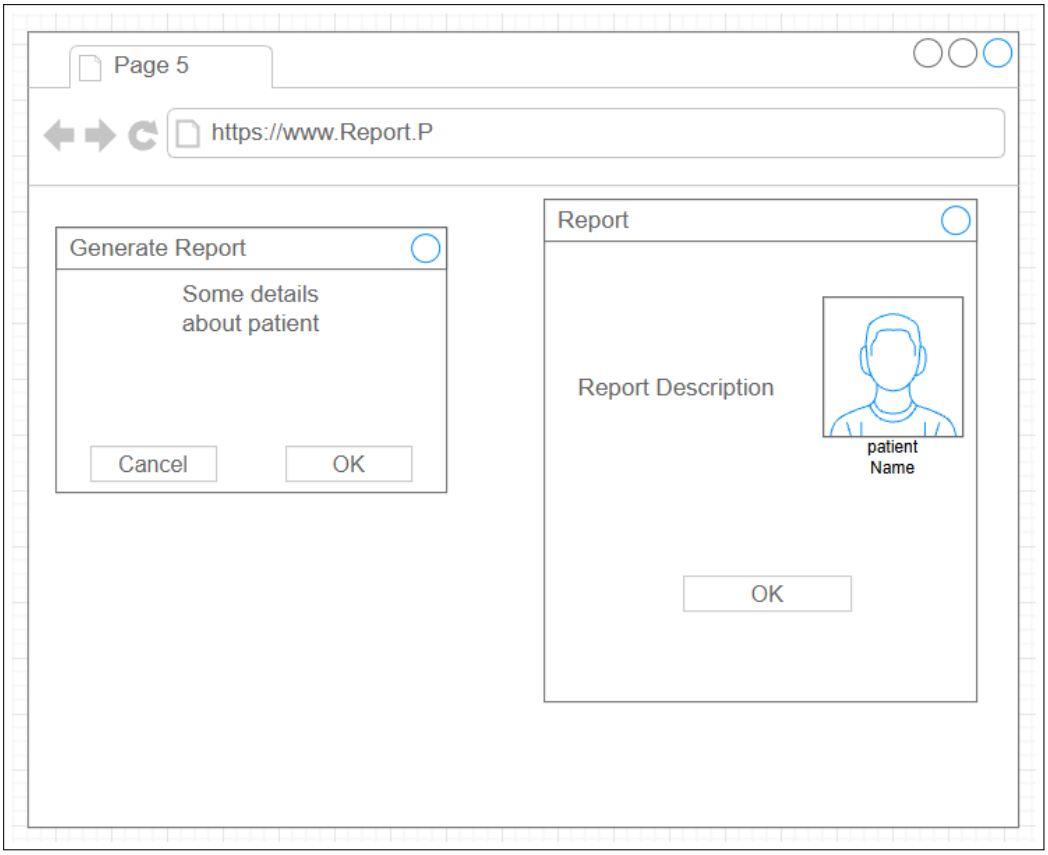


Figure 12: Appointment Booking Wireframe - Features doctor search/filter by specialization, calendar view with available time slots highlighted, appointment details form with reason for visit, and booking confirmation

5. Medical Record View

Figure 13 depicts the medical record interface showing patient history, diagnoses, prescriptions, and vital signs.

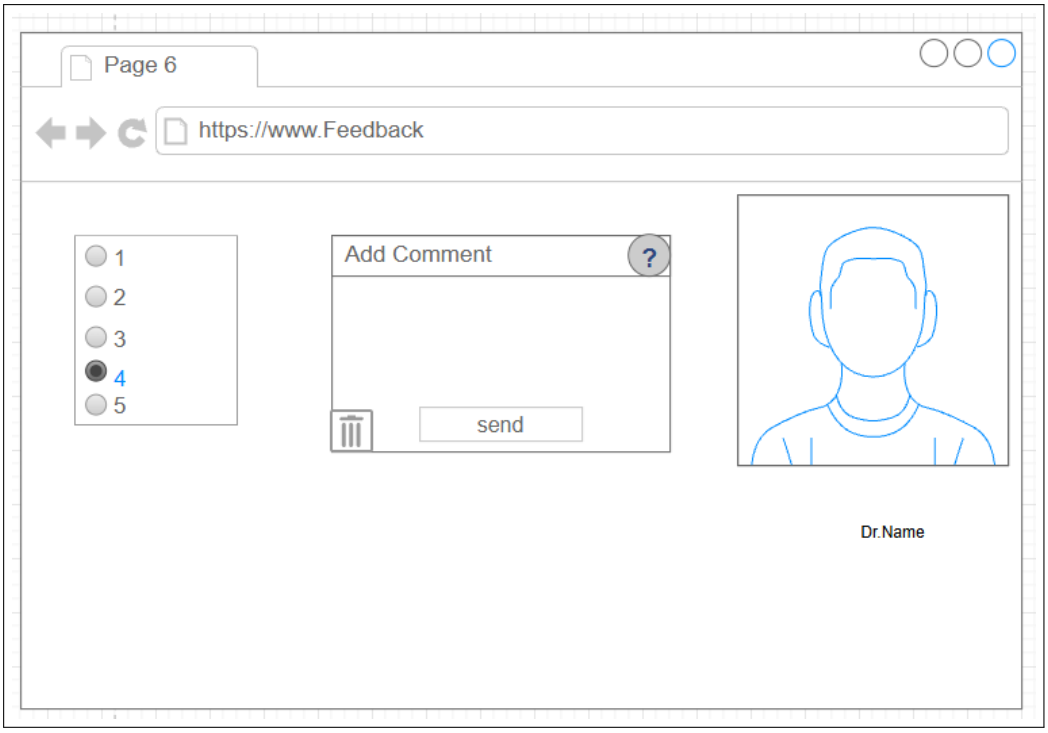


Figure 13: Medical Record Wireframe - Displays patient information header, chronological list of past appointments with diagnoses, prescription details with dosage instructions, vital signs tracking, and doctor's notes

6. Profile Management Page

Figure 14 shows the user profile page where users can update their personal information, contact details, and account settings.

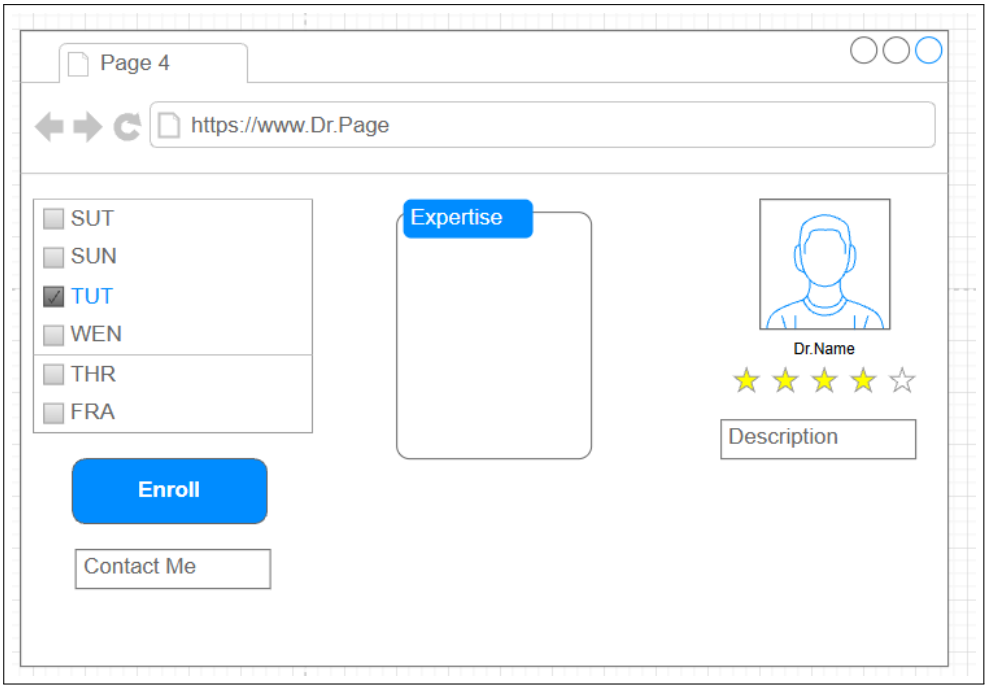


Figure 14: Profile Management Wireframe - Shows editable personal information fields (name, email, phone), password change section, for doctors includes availability schedule management, and profile picture upload

4.4 Data Design

4.4.1 Database Schema / ER Diagram (if applicable)

Clinexa uses a file-based storage system rather than a traditional relational database. Therefore, a traditional ER diagram is not applicable. However, the relationships between data entities are maintained through foreign key references (IDs) stored in the text files.

Data Entity Relationships:

The following describes the key relationships between data entities in the Clinexa system:

- **User to Patient/Doctor:** One-to-one relationship. Each User record can link to either one Patient record or one Doctor record based on the role field.
- **Patient to Appointments:** One-to-many relationship. One patient can have multiple appointments.
- **Doctor to Appointments:** One-to-many relationship. One doctor can have multiple appointments.

- **Appointment to Medical Record:** One-to-one relationship. Each completed appointment generates one medical record.
- **User to Notifications:** One-to-many relationship. One user can receive multiple notifications.
- **Patient to Feedback:** One-to-many relationship. One patient can submit feedback for multiple appointments.
- **Doctor to Feedback:** One-to-many relationship. One doctor can receive feedback from multiple appointments.

4.4.2 File Structure / Data Storage Model

The Clinexa system uses nine text files for data storage, organized in the `clinexa_data/` directory. Each file stores one type of entity with one record per line. Fields within each record are separated by the pipe character (—), and lists within fields use semicolon (;) as a sub-separator.

File Organization:

```
clinexa_data/
 users.txt
 patients.txt
 doctors.txt
 appointments.txt
 medical_records.txt
 notifications.txt
 authentication_logs.txt
 feedback.txt
 reports.txt
```

Data Format Conventions:

- **Primary Separator:** Pipe character (—) separates fields
- **List Separator:** Semicolon (;) separates multiple items within a field
- **Key-Value Separator:** Tilde (˜) separates keys and values in structured data

- **Encoding:** UTF-8 for international character support
- **Line Ending:** Each record is one line terminated with newline
- **Null Values:** Represented as "NULL" or "None" strings

4.4.3 File Schemas

Table 2 presents the complete schema for each data file in the Clinexa system.

File Name	Schema (Fields separated by —)
1. users.txt	userID — username — password — name — email — phone — role — created_date
2. patients.txt	patientID — userID — gender — date_of_birth — blood_type — allergies — medical_history — emergency_contact
3. doctors.txt	doctorID — userID — specialization — license_number — consultation_fee — availability_days — availability_hours — years_experience
4. appointments.txt	appointmentID — patientID — doctorID — appointment_date — appointment_time — status — reason — meeting_link — created_date — updated_date
5. medical_records.txt	recordID — appointmentID — patientID — doctorID — diagnosis — prescription — notes — vital_signs — record_date — record_time
6. notifications.txt	notificationID — recipientID — recipient_type — notification_type — message — sent_date — sent_time — status — channel
7. authentication_logs.txt	logID — userID — username — attempt_date — attempt_time — status — ip_address — failed_attempts_count — is_blocked — block_until
8. feedback.txt	feedbackID — patientID — doctorID — appointmentID — rating — comment — feedback_date — feedback_time — status

File Name	Schema (Fields separated by —)
9. reports.txt	reportID — report_type — generated_by — parameters — generated_date — generated_time — file_path

Table 2: File Schemas for Clinexa Data Storage

Important Implementation Notes:

- Each record occupies exactly ONE line in its respective file
- Fields are separated by the pipe character (—)
- For fields containing multiple values (e.g., allergies, availability_days), items are separated by semicolon (;)
- For key-value pairs within fields (e.g., vital_signs, prescriptions), the tilde character (˜) is used
- All files use UTF-8 encoding to support international characters
- Empty or null values are represented as "NULL" or "None"
- IDs are auto-incremented integers starting from 1
- File locking should be implemented for concurrent write operations

4.4.4 Data Dictionary

This section provides detailed descriptions of each data file, including the purpose of each field, data types, constraints, and example values.

1. users.txt - User Accounts

Field Name	Data Type	Description
userID	Integer	Primary key, unique identifier for each user, auto-incremented
username	String (20)	Unique username for login, alphanumeric only
password	String (255)	Hashed password using Werkzeug security
name	String (100)	Full name of the user
email	String (100)	Valid email address, unique per user
phone	String (15)	Phone number with country code (e.g., +201234567890)
role	String (10)	User role: "patient", "doctor", or "admin"
created_date	Date	Account creation date in YYYY-MM-DD format

Table 3: users.txt Data Dictionary

Example Record:

```
1|ahmed_ragab|hashed_pass123|Ahmed Ragab|ahmed@example.com|
+201234567890|patient|2025-01-15
```

2. patients.txt - Patient Information

Field Name	Data Type	Description
patientID	Integer	Primary key, unique patient identifier
userID	Integer	Foreign key referencing users.txt
gender	String (10)	Patient gender: "male", "female", "other"
date_of_birth	Date	Birth date in YYYY-MM-DD format

Field Name	Data Type	Description
blood_type	String (5)	Blood type (e.g., "O+", "A-", "AB+")
allergies	String (500)	Semicolon-separated list of allergies or "None"
medical_history	String (1000)	Semicolon-separated list of chronic conditions or "None"
emergency_contact	String (150)	Emergency contact name and phone (format: Name:Phone)

Table 4: patients.txt Data Dictionary

Example Record:

```
1|1|male|1995-05-20|O+|Penicillin;Peanuts|Diabetes Type 2;
Hypertension|Fatma Ragab:+201234567891
```

3. doctors.txt - Doctor Information

Field Name	Data Type	Description
doctorID	Integer	Primary key, unique doctor identifier
userID	Integer	Foreign key referencing users.txt
specialization	String (100)	Medical specialization (e.g., "Cardiology", "Pediatrics")
license_number	String (50)	Medical license number, unique
consultation_fee	Float	Consultation fee in local currency
availability_days	String (200)	Semicolon-separated list of available days
availability_hours	String (50)	Working hours in format "HH:MM-HH:MM"
years_experience	Integer	Years of medical practice experience

Table 5: doctors.txt Data Dictionary

Example Record:

```
1|2|Cardiology|LIC123456|500|Sunday;Monday;Wednesday;Thursday|
09:00-17:00|15
```

4. appointments.txt - Appointment Records

Field Name	Data Type	Description
appointmentID	Integer	Primary key, unique appointment identifier
patientID	Integer	Foreign key referencing patients.txt
doctorID	Integer	Foreign key referencing doctors.txt
appointment_date	Date	Appointment date in YYYY-MM-DD format
appointment_time	Time	Appointment time in HH:MM format
status	String (20)	Status: "scheduled", "confirmed", "completed", "cancelled", "no-show"
reason	String (500)	Reason for visit or chief complaint
meeting_link	String (200)	Video consultation link or "NULL" for in-person
created_date	Date	When appointment was booked (YYYY-MM-DD)
updated_date	Date	Last modification date (YYYY-MM-DD)

Table 6: appointments.txt Data Dictionary

Example Record:

```
1|1|1|2025-02-15|10:00|confirmed|Regular checkup|
https://meet.clinexa.com/apt001|2025-01-20|2025-01-21
```

5. medical_records.txt - Medical Records

Field Name	Data Type	Description
recordID	Integer	Primary key, unique record identifier
appointmentID	Integer	Foreign key referencing appointments.txt
patientID	Integer	Foreign key referencing patients.txt
doctorID	Integer	Foreign key referencing doctors.txt
diagnosis	String (1000)	Medical diagnosis or assessment

Field Name	Data Type	Description
prescription	String (2000)	Medications with dosage (format: Drug~Dosage; separated)
notes	String (2000)	Doctor's notes and observations
vital_signs	String (500)	Key-value pairs (format: BP~120/80;HR~72)
record_date	Date	Record creation date (YYYY-MM-DD)
record_time	Time	Record creation time (HH:MM)

Table 7: medical_records.txt Data Dictionary

Example Record:

```
1|1|1|1|Type 2 Diabetes Monitoring|Metformin 500mg~2 times daily;
Insulin Glargine~10 units at bedtime|Patient showing improvement|
BP~120/80;HR~72;Temp~36.8|2025-02-15|10:30
```

6. notifications.txt - Notification Log

Field Name	Data Type	Description
notificationID	Integer	Primary key, unique notification identifier
recipientID	Integer	Foreign key referencing userID in users.txt
recipient_type	String (10)	Type: "patient", "doctor", "admin"
notification_type	String (50)	Type: "appointment_reminder", "appointment_confirmation", "cancellation", "rescheduling"
message	String (1000)	Notification message content
sent_date	Date	When notification was sent (YYYY-MM-DD)
sent_time	Time	Time notification was sent (HH:MM)
status	String (20)	Status: "pending", "sent", "failed", "read"
channel	String (10)	Delivery channel: "email", "sms", "both"

Field Name	Data Type	Description

Table 8: notifications.txt Data Dictionary

Example Record:

```
1|1|patient|appointment_reminder|Your appointment with Dr. Smith is
tomorrow at 10:00 AM|2025-02-14|09:00|sent|both
```

7. authentication_logs.txt - Authentication Logs

Field Name	Data Type	Description
logID	Integer	Primary key, unique log identifier
userID	Integer	Foreign key referencing users.txt (NULL if login failed before user identification)
username	String (20)	Username used in login attempt
attempt_date	Date	Date of login attempt (YYYY-MM-DD)
attempt_time	Time	Time of login attempt (HH:MM:SS)
status	String (20)	Status: "success", "failed", "blocked"
ip_address	String (45)	IP address of login attempt (supports IPv6)
failed_attempts_count	Integer	Cumulative count of failed attempts for this user
is_blocked	Boolean	Whether account is currently blocked (true/false)
block_until	DateTime	When block expires (NULL if not blocked, YYYY-MM-DD HH:MM:SS)

Table 9: authentication_logs.txt Data Dictionary

Example Record:

```
1|1|ahmed_ragab|2025-01-20|14:30:25|success|192.168.1.100|0|
false|NULL
```

8. feedback.txt - Patient Feedback

Field Name	Data Type	Description
feedbackID	Integer	Primary key, unique feedback identifier
patientID	Integer	Foreign key referencing patients.txt
doctorID	Integer	Foreign key referencing doctors.txt
appointmentID	Integer	Foreign key referencing appointments.txt
rating	Integer	Rating from 1 to 5 stars
comment	String (2000)	Patient's feedback comment
feedback_date	Date	Date feedback was submitted (YYYY-MM-DD)
feedback_time	Time	Time feedback was submitted (HH:MM)
status	String (20)	Status: "submitted", "reviewed", "responded"

Table 10: feedback.txt Data Dictionary

Example Record:

```
1|1|1|1|5|Dr. Smith was very professional and took time to
explain everything clearly.|2025-02-15|15:45|submitted
```

9. reports.txt - Generated Reports

Field Name	Data Type	Description
reportID	Integer	Primary key, unique report identifier
report_type	String (50)	Type: "daily_appointments", "patient_history", "doctor_workload", "financial_summary"
generated_by	Integer	Foreign key referencing userID of report generator
parameters	String (500)	Report parameters (format: key˜value; separated)
generated_date	Date	When report was generated (YYYY-MM-DD)

Field Name	Data Type	Description
generated_time	Time	Time report was generated (HH:MM)
file_path	String (255)	Path to generated report file (if saved)

Table 11: reports.txt Data Dictionary

Example Record:

```
1|daily_appointments|2|date~2025-02-15;doctor~all|2025-02-15|
17:00|/reports/daily_2025-02-15.pdf
```

5 Conclusion

5.1 Summary of Design Decisions

This Software Design Document has presented a comprehensive design for the Clinexa Medical System, translating the requirements specified in the SRS into a concrete architectural and detailed design that serves as a blueprint for implementation. The key design decisions made throughout this document include:

1. Architectural Decisions:

- Adoption of a three-tier monolithic architecture providing clear separation between presentation, business logic, and data access layers
- Selection of Flask as the backend framework, leveraging its simplicity and suitability for the project's scope
- Implementation of file-based storage using structured text files with pipe-separated values, meeting project constraints while ensuring data persistence
- Choice of HTML/CSS for the frontend, providing a responsive and accessible user interface without requiring complex JavaScript frameworks

2. Design Pattern Decisions:

- Implementation of the Model-View-Controller (MVC) design pattern to ensure separation of concerns, improve maintainability, and facilitate parallel development
- Clear mapping of all nine system components (User Management, Authentication, Patient Management, Doctor Management, Appointment Management, Medical Records, Notifications, Reports, Search) to the MVC pattern
- Definition of specific responsibilities for Models (business logic and data persistence), Views (presentation), and Controllers (request handling and flow control)

3. Data Design Decisions:

- Creation of nine structured text files for storing different types of data entities

- Use of pipe (—) as primary field separator, semicolon (;) for list items, and tilde (˜) for key-value pairs
- Implementation of foreign key references through ID fields to maintain relationships between entities
- UTF-8 encoding for international character support
- Comprehensive data dictionaries defining field types, constraints, and example values

4. User Interface Design Decisions:

- Creation of role-specific dashboards for patients and doctors with relevant features and information
- Design of intuitive appointment booking interface with doctor search, calendar view, and availability checking
- Development of comprehensive medical record views displaying patient history, diagnoses, prescriptions, and vital signs
- Implementation of clear navigation structure and consistent visual design across all pages

5. Security Design Decisions:

- Password hashing using Werkzeug security utilities
- Implementation of failed login attempt tracking with automatic account blocking after three attempts
- Role-based access control ensuring users can only access appropriate features and data
- Session management for maintaining authenticated user state

5.2 Design Phase Outputs

The design phase has produced the following key deliverables that will guide the implementation phase:

- **Architectural Diagrams:** System architecture diagram showing the three-tier monolithic design with clear layer separation and component interactions
- **MVC Component Mapping:** Comprehensive mapping table showing how each of the nine system components is organized according to the MVC pattern
- **Sequence Diagrams:** Five detailed sequence diagrams illustrating the flow of operations for:
  - User login and authentication
  - Appointment booking process
  - Medical record management
  - Report generation
  - Search and filtering operations
- **UI/UX Wireframes:** Six wireframes/mockups covering:
  - Login page
  - Patient dashboard
  - Doctor dashboard
  - Appointment booking interface
  - Medical record view
  - Profile management page
- **Data Specifications:** Complete file schemas and data dictionaries for all nine data files, including field definitions, data types, separators, constraints, and example records
- **Technology Stack Documentation:** Detailed specification of all technologies, tools, frameworks, and external services to be used in implementation

5.3 Alignment with Requirements

The design presented in this document fully aligns with and addresses all functional and non-functional requirements specified in the Software Requirements Specification:

Functional Requirements Coverage:

- User authentication and authorization with role-based access control
- Patient registration and profile management
- Doctor profile management with availability scheduling
- Appointment booking, cancellation, and rescheduling with availability checking
- Medical record creation, retrieval, and management
- Automated notification system for reminders and confirmations via email and SMS
- Report generation for appointments, patient history, and doctor workload
- Search functionality for doctors, patients, and appointments
- Feedback and rating system for patient-doctor interactions

Non-Functional Requirements Coverage:

- **Usability:** Intuitive interface design with clear navigation and role-specific dashboards
- **Reliability:** Robust error handling, data validation, and file locking mechanisms to ensure system stability
- **Performance:** Efficient file-based storage with optimized reading/writing operations
- **Security:** Password hashing, failed login tracking, account blocking, and role-based access control
- **Maintainability:** MVC pattern, modular design, and clear separation of concerns facilitating future updates
- **Compatibility:** Browser-based interface compatible with Chrome and Firefox

5.4 Next Steps

With the design phase complete, the project is now ready to proceed to the implementation phase. The development team should:

1. **Review and Approval:** Present this design document to stakeholders and course instructors for review and approval before beginning implementation.

2. **Environment Setup:** Set up the development environment including Python, Flask, version control, and necessary tools.

3. **Implementation Planning:** Create a detailed implementation schedule assigning specific components to team members based on the MVC structure defined in this document.

4. **Iterative Development:** Implement components incrementally, starting with core functionality (authentication, user management) and progressively adding features (appointments, medical records, notifications, reports).

5. **Testing:** Conduct unit testing, integration testing, and user acceptance testing as components are developed to ensure they meet the specified requirements.

6. **Documentation:** Maintain code documentation and update this design document if significant design changes are necessary during implementation.

This Software Design Document provides a solid foundation for building a secure, maintainable, and user-friendly healthcare management system that meets all specified requirements while adhering to project constraints and software engineering best practices.